Fitting N-mixture models

Analysing Ecological Data with Detection Error

Péter Sólymos

2026-04-20

Table of contents

Preamble 1

Simulating count data 2

Observation error for counts 3

Count data with multiple visits 3

Fitting N-mixture models 5

N-mixture likelihood 7

Field data analysis 10

Preamble

```
suppressPackageStartupMessages({
    library(unmarked)
    library(detect)
    library(dplyr)
    library(ggplot2)
    theme_set(theme_light())
})
set.seed(1234)
```

Simulating count data

Simulating counts is similar to occupancy. We need a count distribution. The most basic count distribution is Poisson which has 1 parameter, λ, which is the mean. The variance also happens to equal the mean. Use the `rpois()` function in R to generate random numbers from the poisson distribution. Here, N_i $(i = 1, \ldots, n)$ is the abundance (number of individuals) at the ith location.

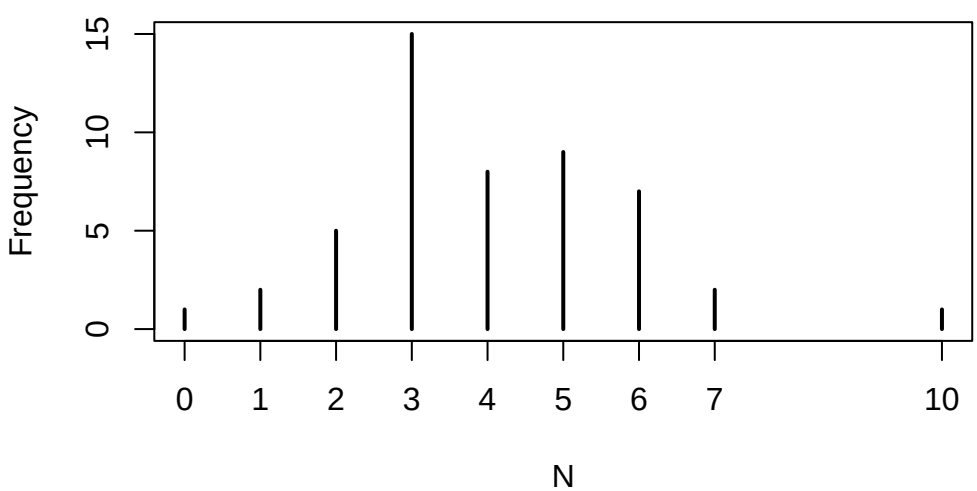
```
n <- 50
n2 <- 500
p.true <- 0.6

lambda.true <- 4.2

N <- rpois(n = n, lambda = lambda.true)

table(N)
```

```
N
 0  1  2  3  4  5  6  7 10
 1  2  5 15  8  9  7  2  1
```

```
summary(N)
```

```
   Min. 1st Qu.  Median    Mean 3rd Qu.    Max. 
      0       3       4       4       5      10 
```

```
table(N) |> plot(ylab = "Frequency")
```



Observation error for counts

Here is how we introduce observation error to count data:

- select a location,
- take the count N_i,
- for each individual $(1, 2, \dots, N_i)$, use the Bernoulli distribution with probability p to determine if the individual was detected (1) or not (0)

The number of individuals detected (Y_i) is less than or equal to N_i: $Y_i \leq N_i$. For example, if $N_i = 4$, Y_i can be 0, 1, 2, 3, or 4.

To express things mon concisely, we can also use a Binomial distribution with N_i as the number of trials and probability p:

```
Y <- rbinom(n = n, size = N, prob = p.true)

print(table(N = N, Y = Y), zero.print = ".")
```

```
    Y
N    0 1 2 3 4 5 7
  0  1 . . . . . .
  1  . 2 . . . . .
  2  2 2 1 . . . .
  3  . 4 5 6 . . .
  4  . 2 2 3 1 . .
  5  . . 3 3 3 . .
  6  . . . . 2 5 .
  7  . . . . . 2 .
  10 . . . . . . 1
```

Count data with multiple visits

When visiting each site T times, the observed data will be organized in a n by T matrix as how we saw it for occupancy:

```
Y <- matrix(NA, n, T)
for (t in 1:T) {
    Y[, t] <- rbinom(n = n, size = N, prob = p.true)
}
```

```r
# alternatively
# Y <- replicate(T, rbinom(n = n, size = N, prob = p.true))
```

Multiple-visit count model have similar assumptions as occupancy models, most importantly that $N_{it} = N_i$. This condition is also referred to as the closed population assumption, i.e. there is no immigration, emigration, birth, or death between visits.

```r
data.frame(N = N, Visit = Y) |> head()
```

```
  N Visit
1 2     1
2 5     5
3 5     3
4 5     4
5 6     5
6 5     4
```

As we saw before, we can use the `Y_max` to fit the naive count model, but instead of the `plogis()` function, we use `log()` as the inverse of the logarithmic link used for Poisson GLM:

```r
Y_max <- apply(Y, 1, max)
m4 <- glm(Y_max ~ 1, family = poisson)
coef(m4)
```

```
(Intercept)
  0.8329091
```

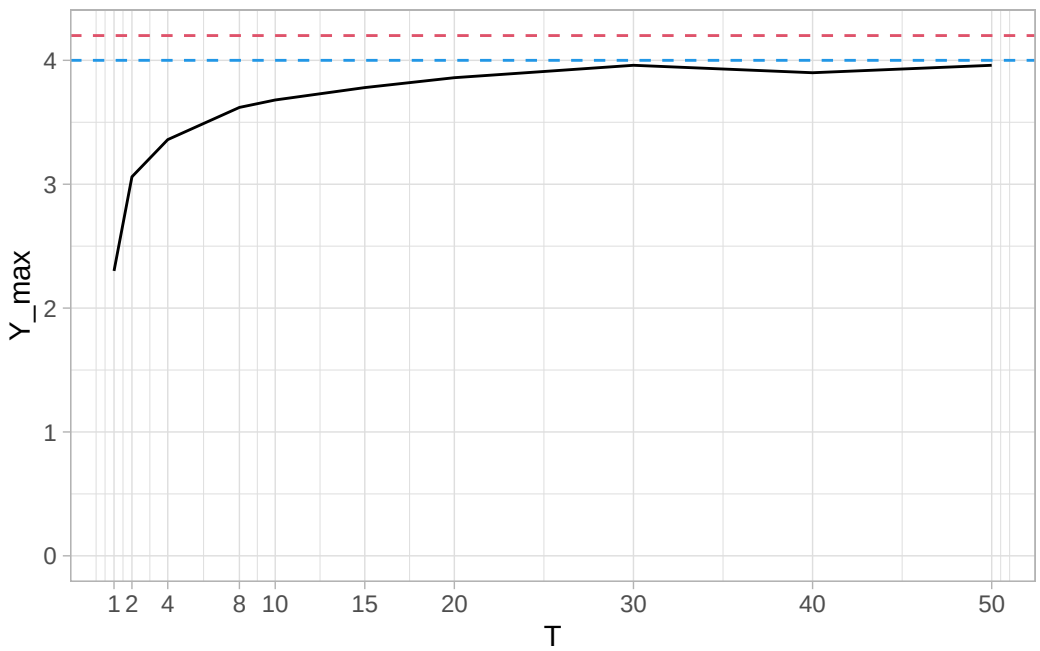
```r
exp(coef(m4))
```

```
(Intercept)
        2.3
```

With enough visits, tha `Y_max` will approach the latent variable `N`:

```r
N2 <- rpois(n = n2, lambda = lambda.true)
Tvals <- c(1, 2, 4, 8, 10, 15, 20, 30, 40, 50)

data.frame(
    T = Tvals,
```

```
    Y_max = sapply(Tvals, \(T) {
        mean(apply(
            replicate(T, rbinom(n = n, size = N, prob = p.true)),
            1, max
        ))
    })
) |> ggplot(aes(x = T, y = Y_max)) +
    geom_hline(yintercept = lambda.true, lty = 2, col = 2) +
    geom_hline(yintercept = mean(N), lty = 2, col = 4) +
    geom_line() +
    ylim(0, NA) +
    scale_x_continuous(breaks = Tvals) +
    theme_light()
```



Fitting N-mixture models

Fit the multiple-visit count model, the so called N-mixture model, using the `pcount()` function of the unmarked R package.

The `unmarked::unmarkedFramePCount()` function is used to organize the count data:

```r
umf <- unmarked::unmarkedFramePCount(y = Y)
summary(umf)
```

```
unmarkedFrame Object

50 sites
Maximum number of observations per site: 1
Mean number of observations per site: 1
Sites with at least one detection: 43

Tabulation of y observations:
 0  1  2  3  4  5  6
 7  8 16  7  7  4  1
```

```r
m5 <- unmarked::pcount(~1 ~ 1, umf, K = 50)
m5
```

```
Call:
unmarked::pcount(formula = ~1 ~ 1, data = umf, K = 50)

Warning in sqrt(diag(vcov(obj, fixedOnly = fixedOnly))): NaNs produced

Abundance (log-scale):
 Estimate  SE   z P(>|z|)
     2.12 NaN NaN     NaN

Warning in sqrt(diag(vcov(obj, fixedOnly = fixedOnly))): NaNs produced

Detection (logit-scale):
 Estimate  SE   z P(>|z|)
   -0.964 NaN NaN     NaN

AIC: 185.6474
Number of sites: 50

Warning in sqrt(diag(v)): NaNs produced

Warning: Large or missing SE values. Be very cautious using these results.
```

```r
plogis(coef(m5, type = "det"))
```

```
   p(Int)
0.2760193
```

```r
exp(coef(m5, type = "state"))
```

```
lam(Int)
8.332899
```

> References
>
> Royle, 2004. N-mixture models for estimating population size from spatially replicated counts. Biometrics, 60:108–115. Fulltext, DOI 10.1111/j.0006-341X.2004.00142.x

The `K` argument of the `pcount()` function is the upper index of integration for N-mixture. This will make more sense once we take a look at the likelihood function in the next section.

N-mixture likelihood

The likelihood function for the N-mixture model can be written as:

$$L(p, \lambda; y_{1,1}, \dots, y_{n,T}) = \prod_{i=1}^{n} \sum_{N_i = Y_{max,i}}^{K} \prod_{t=1}^{T} e^{-\lambda_i} \frac{\lambda_i^{N_i}}{N_i!} \left(\begin{array}{c} N_i \\ Y_{it} \end{array} \right) p^{Y_{it}} (1-p)^{N_i - Y_{it}}$$

This distribution is a mixture of Poisson and Binomial distributions.

The K value strictly speaking should be ∞, but the estimates won't change much as long as K is large enough relative to the maximum of N_i. Of course for real data we do not know what the values of N_i are. The numerical integration (summation) for site i goes from $Y_{max,i}$ (`Y_max`) to K, because we know that $Y_{it} \leq N_i$.

We can write this as a log likelihood function:

```r
logL_fun_pcount_R <- function(Y, p, lambda, n, T, Y_max, K) {
    L <- rep(NA, n)
    for (i in 1:n) {
        S <- 0
        for (Nit in Y_max[i]:K) {
            v <- 0
```

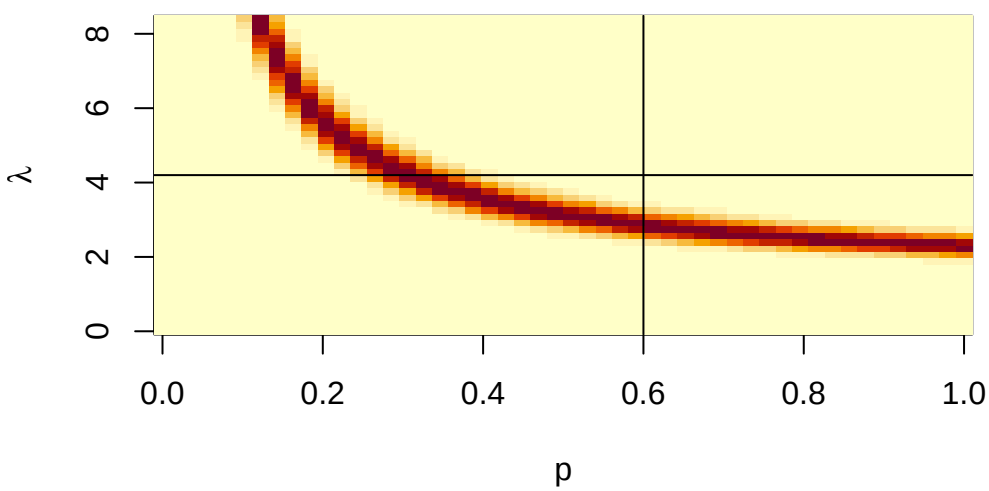
```r
            for (j in 1:T) {
                v <- v + dbinom(Y[i, j], Nit, p, log = TRUE) +
                    dpois(Nit, lambda, log = TRUE)
            }
            S <- S + exp(v)
        }
        L[i] <- S
    }
    sum(log(L))
}

# using C code from unmarked - much faster
logL_fun_pcount_C <- function(Y, p, lambda, n, T, Y_max, K = 50) {
    nll <- unmarked:::nll_pcount(
        beta = c(log(lambda), qlogis(p)),
        n_param = c(1, 1, 0),
        y = Y,
        X = matrix(1, n, 1),
        V = matrix(1, n * T, 1),
        X_offset = rep(0, n),
        V_offset = matrix(1, n * T, 1),
        K = K,
        Kmin = Y_max,
        mixture = 1,
        threads = 1
    )
    -nll
}
```

```r
g <- 50
grid <- expand.grid(
    p = seq(0, 1, length.out = g),
    lambda = seq(0, lambda.true * 2, length.out = g),
    logL = NA
)

logL_fun_pcount <- logL_fun_pcount_C
for (i in 1:nrow(grid)) {
    grid$logL[i] <- logL_fun_pcount(
        Y = Y,
        p = grid$p[i],
        lambda = grid$lambda[i],
```

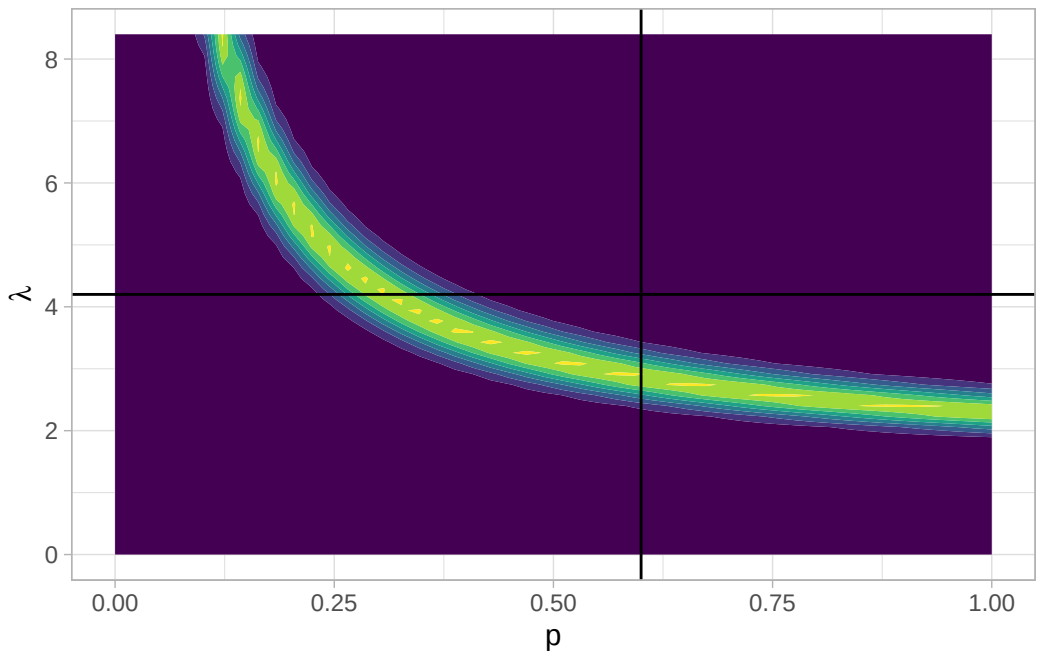
```
        n = n, T = T, Y_max = Y_max, K = 50
    )
}
grid$logL[is.infinite(grid$logL)] <- NA
```

The images showing the likelihood surface:

```
image(
    list(
        x = unique(grid$p),
        y = unique(grid$lambda),
        z = matrix(exp(grid$logL), sqrt(nrow(grid)))
    ),
    xlab = "p",
    ylab = expression(lambda)
)
abline(h = lambda.true, v = p.true, col = 1, lwd = 1)
```




```
grid |> ggplot(aes(x = p, y = lambda, z = exp(logL))) +
    geom_contour_filled(show.legend = FALSE) +
    geom_hline(yintercept = lambda.true) +
    geom_vline(xintercept = p.true) +
    xlab("p") +
    ylab(expression(lambda)) +
    theme_light()
```




```r
grid[which.max(grid$logL), ]
```

```
             p   lambda      logL
1022 0.4285714 3.428571 -90.82372
```

> **Note**
>
> We will circle back to these plots later, feel free to explore it with different settings of n, T, p, and λ.

Field data analysis

We will use the ABMI data, which has 3 equal (3.3 min) time intervals for each point count visit. We use the point level data, but only for the center point, so we can keep sample size small and run the code quickly. When analyzing such data sets for real, we would use all the data and not subset it like this.

```r
if (!interactive()) {
    d <- readRDS("../data/ab_birds_site_year_pt.rds")
} else {
    d <- readRDS("data/ab_birds_site_year_pt.rds")
```

```r
}

spp <- "yellowrumped_warbler"

x <- data.frame(
    y1 = ifelse(d$visits$visit1[, spp] > 0, 1, 0),
    y2 = ifelse(d$visits$visit2[, spp] > 0, 1, 0),
    y3 = ifelse(d$visits$visit3[, spp] > 0, 1, 0),
    n1 = d$visits$visit1[, spp],
    n2 = d$visits$visit2[, spp],
    n3 = d$visits$visit3[, spp],
    d$vars
)
x$yday <- as.POSIXlt(x$fielddate)$yday / 365
x$hr <- x$fieldtime / 24
x$y_max <- pmax(x$y1, x$y2, x$y3)
x$n_max <- pmax(x$n1, x$n2, x$n3)
x <- x |>
    filter(
        hr > 0.1,
        hr < 0.6,
        yday > 0.37,
        yday < 0.5
    )
x0 <- x[x$point == 1, ]
```

Naive abundance estimates for the 3 visits:

```r
m_v1 <- glm(n1 ~ 1, data = x0, family = poisson)
coef(m_v1)
```

```
(Intercept)
 -0.9800302
```

```r
exp(coef(m_v1))
```

```
(Intercept)
  0.3752998
```

```r
mean(x0$n1)
```

```
[1] 0.3752998
```

```r
m_v2 <- glm(n2 ~ 1, data = x0, family = poisson)
coef(m_v2)
```

```
(Intercept)
 -0.9928921
```

```r
exp(coef(m_v2))
```

```
(Intercept)
  0.3705036
```

```r
mean(x0$n2)
```

```
[1] 0.3705036
```

```r
m_v3 <- glm(n3 ~ 1, data = x0, family = poisson)
coef(m_v3)
```

```
(Intercept)
   -1.02914
```

```r
exp(coef(m_v3))
```

```
(Intercept)
  0.3573141
```

```r
mean(x0$n3)
```

```
[1] 0.3573141
```

Use the maximum count to fit the naive model:

```r
m_max <- glm(n_max ~ 1, data = x0, family = poisson)
coef(m_max)
```

```
(Intercept)
 -0.6859787
```

```r
exp(coef(m_max))
```

```
(Intercept)
  0.5035971
```

```r
mean(x0$n_max)
```

```
[1] 0.5035971
```

N-mixture model using the `pcount()` function:

```r
# here N-mix version using pcount
umf <- unmarked::unmarkedFramePCount(y = x0[, c("n1", "n2", "n3")])
summary(umf)
```

```
unmarkedFrame Object

834 sites
Maximum number of observations per site: 3
Mean number of observations per site: 3
Sites with at least one detection: 264

Tabulation of y observations:
   0    1    2    3    4
1884  387  170   51   10
```

```r
m_mv <- unmarked::pcount(~1 ~ 1, umf, K = 100)
m_mv
```

```
Call:
unmarked::pcount(formula = ~1 ~ 1, data = umf, K = 100)
```

```
Abundance (log-scale):
 Estimate     SE     z  P(>|z|)
   -0.634 0.0494 -12.8 1.19e-37

Detection (logit-scale):
 Estimate    SE  z  P(>|z|)
    0.814 0.074 11 3.73e-28

AIC: 2898.027
Number of sites: 834
```

```r
plogis(coef(m_mv, type = "det"))
```

```
   p(Int)
0.6929904
```

```r
exp(coef(m_mv, type = "state"))
```

```
 lam(Int)
0.5305777
```

Use covariates to model abundance and detection:

```r
umf2 <- unmarked::unmarkedFramePCount(
    y = x0[, c("n1", "n2", "n3")],
    siteCovs = x0[, c("pconif", "thf")],
    obsCovs = list(
        hr = x0[, c("hr", "hr", "hr")],
        yday = x0[, c("yday", "yday", "yday")]
    )
)
summary(umf2)
```

```
unmarkedFrame Object

834 sites
Maximum number of observations per site: 3
Mean number of observations per site: 3
Sites with at least one detection: 264
```

```
Tabulation of y observations:
   0    1    2    3    4
1884  387  170   51   10

Site-level covariates:
     pconif            thf
 Min.   :0.0000   Min.   :0.00000
 1st Qu.:0.0000   1st Qu.:0.00000
 Median :0.0000   Median :0.06333
 Mean   :0.2740   Mean   :0.34896
 3rd Qu.:0.6036   3rd Qu.:0.86650
 Max.   :1.0000   Max.   :1.00000

Observation-level covariates:
       hr              yday
 Min.   :0.1493   Min.   :0.3890
 1st Qu.:0.2111   1st Qu.:0.4082
 Median :0.2257   Median :0.4192
 Mean   :0.2516   Mean   :0.4273
 3rd Qu.:0.2562   3rd Qu.:0.4493
 Max.   :0.5271   Max.   :0.4822
```

```r
m_mv2 <- unmarked::pcount(
    ~ hr + I(hr^2) + yday ~ pconif + I(pconif^2) + thf,
    umf2,
    K = 100
)
m_mv2
```

```
Call:
unmarked::pcount(formula = ~hr + I(hr^2) + yday ~ pconif + I(pconif^2) +
    thf, data = umf2, K = 100)

Abundance (log-scale):
            Estimate    SE     z  P(>|z|)
(Intercept)   -0.981 0.112 -8.78 1.57e-18
pconif         3.568 0.537  6.65 3.00e-11
I(pconif^2)   -2.361 0.515 -4.58 4.63e-06
thf           -1.632 0.234 -6.98 2.92e-12

Detection (logit-scale):
            Estimate    SE       z P(>|z|)
```

```
(Intercept)    3.700  2.01  1.8442  0.0652
hr             0.439  9.56  0.0459  0.9634
I(hr^2)        0.751 15.13  0.0496  0.9604
yday          -7.428  3.69 -2.0151  0.0439

AIC: 2578.76
Number of sites: 834

Warning: Large or missing SE values. Be very cautious using these results.
```

```r
m_mv@AIC
```

```
[1] 2898.027
```

```r
m_mv2@AIC
```

```
[1] 2578.76
```

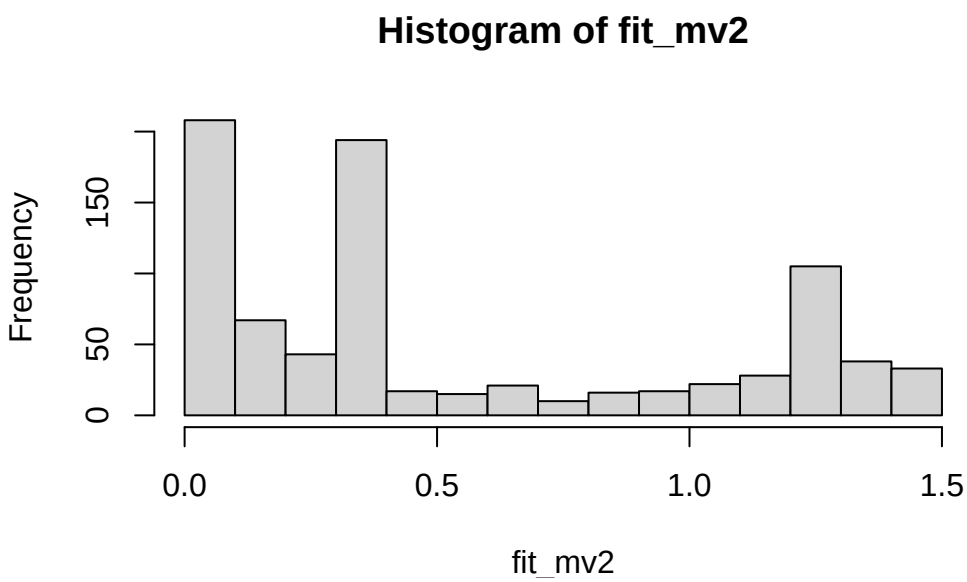
```r
summary(fitted(m_mv2))
```

```
       V1                V2                V3
 Min.   :0.04181   Min.   :0.04181   Min.   :0.04181
 1st Qu.:0.06299   1st Qu.:0.06299   1st Qu.:0.06299
 Median :0.23807   Median :0.23807   Median :0.23807
 Mean   :0.36495   Mean   :0.36495   Mean   :0.36495
 3rd Qu.:0.71320   3rd Qu.:0.71320   3rd Qu.:0.71320
 Max.   :1.04481   Max.   :1.04481   Max.   :1.04481
```

```r
colMeans(umf2@y)
```

```
       n1        n2        n3
0.3752998 0.3705036 0.3573141
```

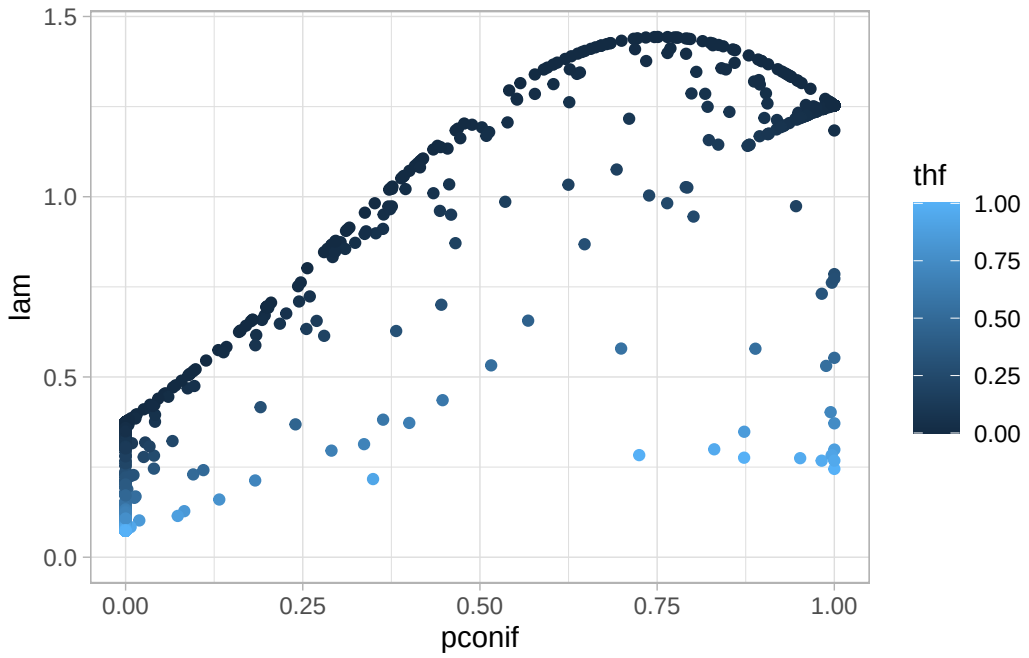
```r
X <- model.matrix(m_mv2@formula[-2], x0)
fit_mv2 <- exp(X %*% coef(m_mv2, type = "state"))
hist(fit_mv2)
```



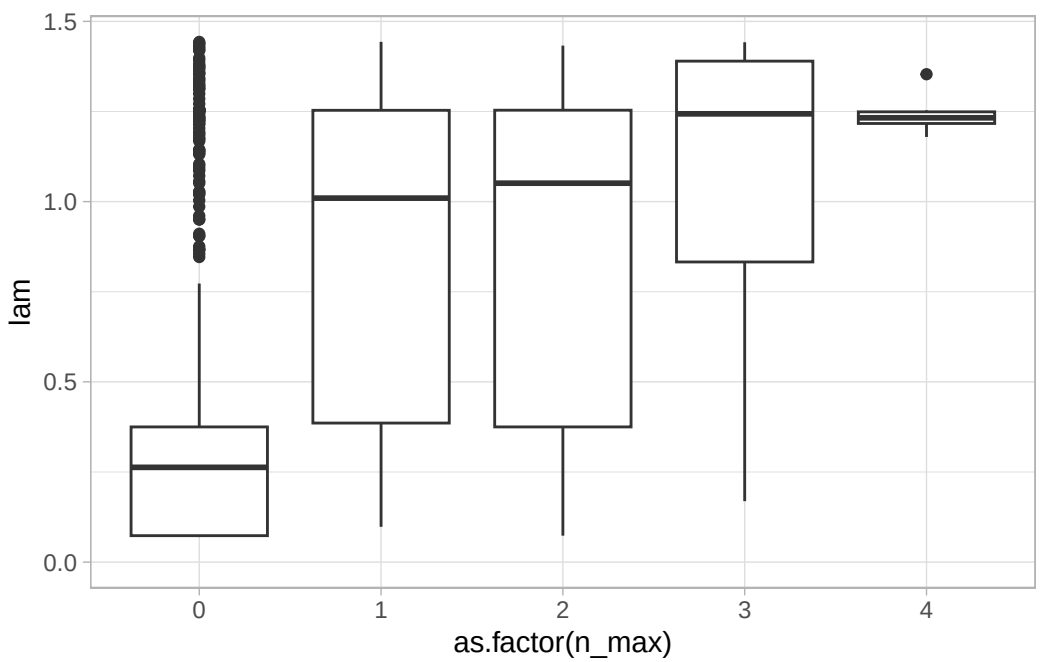

```r
summary(fit_mv2)
```

```
       V1
 Min.   :0.07331
 1st Qu.:0.10041
 Median :0.36936
 Mean   :0.54835
 3rd Qu.:1.07995
 Max.   :1.44325
```

```r
data.frame(x0, lam = fit_mv2) |>
    ggplot(aes(x = pconif, y = lam, col = thf)) +
    geom_point() +
    ylim(0, NA)
```

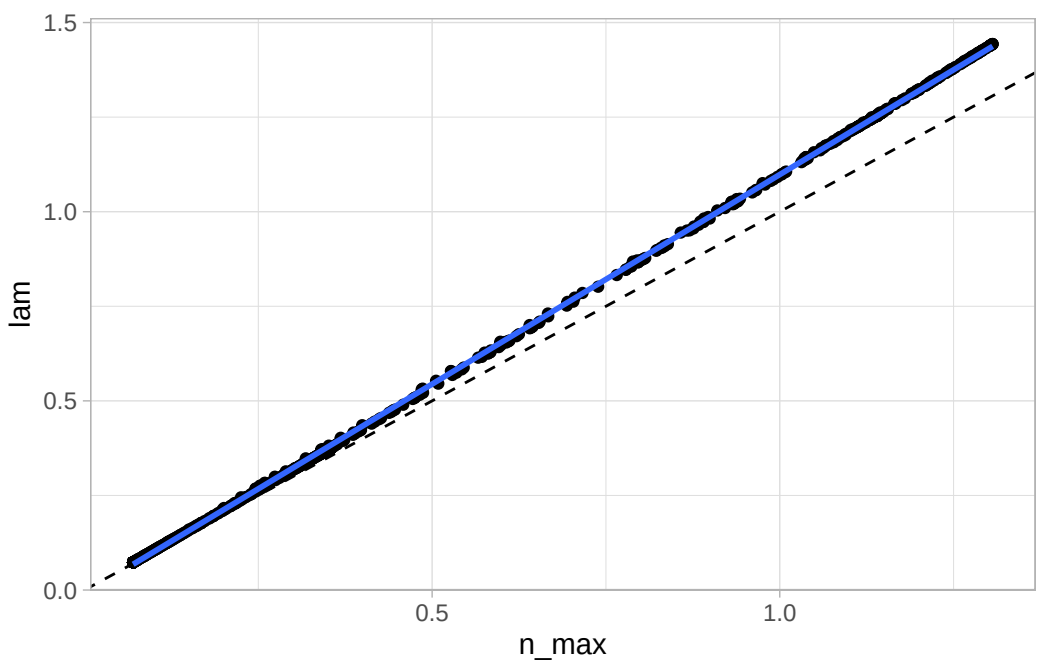



```
data.frame(x0, lam = fit_mv2) |>
    ggplot(aes(x = as.factor(n_max), y = lam)) +
    geom_boxplot() +
    ylim(0, NA)
```



```r
m_max2 <- glm(n_max ~ pconif + I(pconif^2) + thf, data = x0, family = poisson)

data.frame(lam = fit_mv2, n_max = fitted(m_max2)) |>
    ggplot(aes(x = n_max, y = lam)) +
    geom_point() +
    geom_abline(lty = 2) +
    geom_smooth(method = "lm", se = FALSE)
```

```
`geom_smooth()` using formula = 'y ~ x'
```




```r
mean(fit_mv2 / fitted(m_max2))
```

```
[1] 1.070923
```

```r
Z <- model.matrix(m_mv2@formula[-3], x0)
head(Z)
```

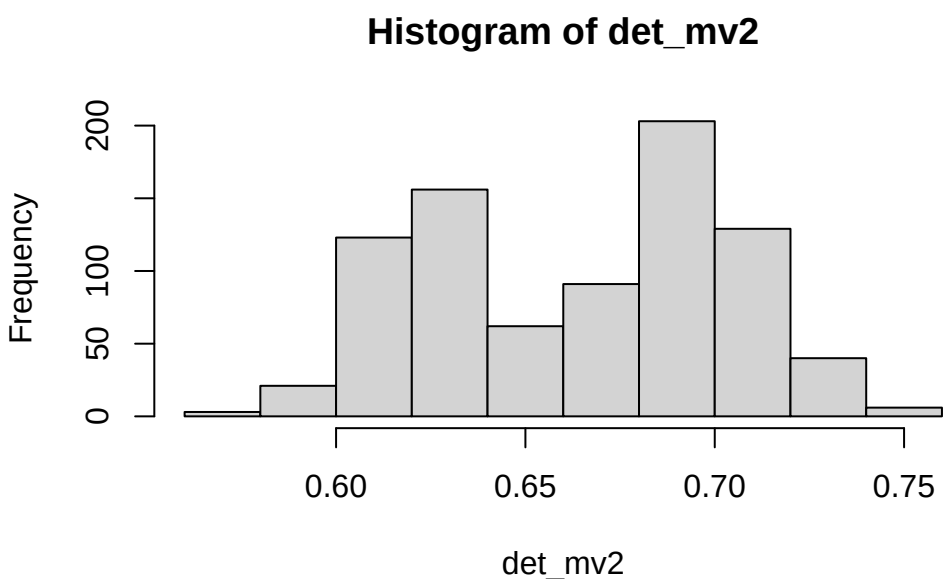
```
            (Intercept)        hr    I(hr^2)      yday
1_2009_1              1 0.2354167 0.05542101 0.4410959
1001_2011_1           1 0.2437500 0.05941406 0.4465753
1002_2011_1           1 0.2465278 0.06077595 0.4356164
```

```
1003_2011_1           1 0.3888889 0.15123457 0.4383562
1007_2008_1           1 0.2666667 0.07111111 0.4054795
1008_2008_1           1 0.2020833 0.04083767 0.4164384
```

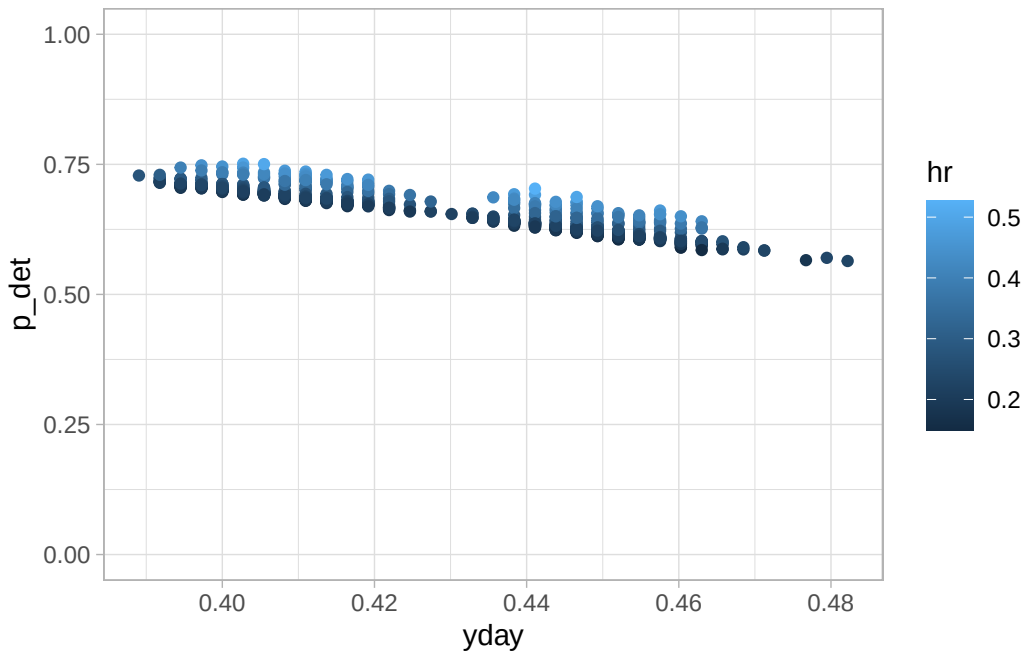
```r
det_mv2 <- plogis(Z %*% coef(m_mv2, type = "det"))
summary(det_mv2)
```

```
       V1
 Min.   :0.5642
 1st Qu.:0.6273
 Median :0.6762
 Mean   :0.6643
 3rd Qu.:0.6975
 Max.   :0.7512
```

```r
hist(det_mv2)
```




```r
data.frame(x0, p_det = det_mv2) |>
    ggplot(aes(x = yday, y = p_det, col = hr)) +
    geom_point() +
    ylim(0, 1)
```



What do we get when only using 2 visits?

```
umf_v2 <- unmarked::unmarkedFramePCount(y = x0[, c("n1", "n2")])
m_mv_v2 <- unmarked::pcount(~1 ~ 1, umf_v2, K = 100)
m_mv_v2
```

```
Call:
unmarked::pcount(formula = ~1 ~ 1, data = umf_v2, K = 100)

Abundance (log-scale):
 Estimate    SE     z  P(>|z|)
   -0.712 0.053 -13.4 3.54e-41

Detection (logit-scale):
 Estimate    SE    z  P(>|z|)
     1.15 0.114 10.1 4.82e-24

AIC: 2172.117
Number of sites: 834
```

```
plogis(coef(m_mv_v2, type = "det"))
```

```
  p(Int)
0.759897
```

```r
exp(coef(m_mv_v2, type = "state"))
```

```
 lam(Int)
0.4907264
```

How about only using 1 visit?

```r
umf_v1 <- unmarked::unmarkedFramePCount(y = x0[, c("n1"), drop = FALSE])
m_mv_v1 <- unmarked::pcount(~1 ~ 1, umf_v1, K = 100)
m_mv_v1
```

```
Call:
unmarked::pcount(formula = ~1 ~ 1, data = umf_v1, K = 100)

Abundance (log-scale):
 Estimate  SE        z P(>|z|)
   -0.366 336 -0.00109   0.999

Detection (logit-scale):
 Estimate  SE        z P(>|z|)
    0.166 733 0.000226       1

AIC: 1405.514
Number of sites: 834

Warning: Large or missing SE values. Be very cautious using these results.
```

```r
plogis(coef(m_mv_v1, type = "det"))
```

```
   p(Int)
0.5413588
```

```r
exp(coef(m_mv_v1, type = "state"))
```

```
 lam(Int)
0.6932552
```

Single visit estimates for the constant abundance model are not reliable.

There are tools out there (e.g. data cloning) that you can used to see if the model you are fitting is identifiable or not.

Exercise: Try `"P"`, `"NB"`, `"ZIP"` mixtures for `pcount()`.